

INO Security Audit IV

This was the strongest remediation round so far. Every Critical and every High from the last report is genuinely closed, and the quality of the fixes improved markedly. Three new problems appeared, one of which is the previous Critical wearing a better disguise.

4 Sep 2026 fc3aaa5 → 6390fdf · 5 commits 508 of 508 tests pass

OVERALL SCORE

6/10

up from 5/10 · 4 at pass 1

Closer, still not submittable

Eleven confirmed defects closed.
Three new ones, plus two
deployment blockers and an
untouched privacy surface.

Auth & session		7 ▲1
Sharing		6 ▲1
Platform & build		6 ▲1
Backend & RLS		5 ▼1
Crypto & local data		5 –
Privacy & Play		5 ▲1

What genuinely got fixed

All three Criticals and all eight Highs from the last report verify as properly closed. I re-read each in the code rather than trusting commit messages, and several were fixed the hard, correct way rather than the quick way.

- **Account deletion finally works.** Every column now matches its defining table, the blanket error handler that hid failures is gone, and it reaches the final delete of the auth user. Third attempt, and this one type-checks end to end.
- **The session is finally off plaintext disk.** Access and refresh tokens now live in Keystore/Keychain, with a migration that does not sign existing users out and reads that degrade to signed-out rather than crashing. This was open across all three previous audits.
- **The two-factor policies were fixed the right way.** Rather than adding new policies beside the old ones, this round drops the factory-generated policies by the exact

names the factory generates, so the requirement actually binds instead of being OR'd into irrelevance.

- **The vault admin can no longer depose the owner.** The old policy is dropped and replaced with the condition in both clauses, including the one where the hole actually was.
- **The consent gate is back and better placed than before** — enforced in shared post-auth routing, defensively on shell entry, on cold start, and on the offline path, with a real audit table and regression tests.
- **The package ID is restored and consistent** across all eight locations, confirmed by running the manifest merger rather than reading files.
- **Legacy protected shares are now revoked rather than unlocked**, with the edge function failing closed independently — the fix I was most expecting to be hollow, and it is not.
- **The lockout functions are properly revoked**, the share site compiles again, and the offline encryption key now fails closed in both directions.

One pattern is worth more than any single finding

The iOS deep-link file now has the right shape and the wrong value:

```
apple-app-site-association:6    "appID": "TEAMID.com.ino.app"
```

`TEAMID` is a literal placeholder, not your Apple Team ID, and the Xcode project contains no team ID anywhere. So iOS Universal Links cannot activate at all. What makes this worth highlighting is the test that guards it:

```
share_security_hardening_test.dart:143  
expect(detail['appID'], contains('com.ino.app'));
```

`contains` is a substring check, so it passes on the placeholder. The suite is green, the file looks fixed, and the feature is dead. Tests are now being written — which is real progress — but several assert that a fix is *present* rather than that it *works*. That is the same failure mode as the previous three rounds, now one level up the stack.

Critical

C1

"Forgot passphrase" no longer deletes the vault — it double-encrypts it

CRITICAL **NEW** **verified**

password_vault_screen.dart:158 · password_store.dart:128, :269-273, toRow:172

The deletion is genuinely gone and both new guards work. But the data loss did not go away — it changed shape and became harder to spot. I traced every link:

The unlock sheet carrying "Forgot passphrase" is only shown when the vault is locked. Loading the store while locked skips the decrypt branch, so every entry is held with its *ciphertext* sitting in the password field, indistinguishable from a plaintext password. The list is therefore non-empty, so the new empty-list guard passes and the dialog reports an honest count — which makes the user *more* likely to proceed than when it wrongly said zero. The reset then replaces the salt and re-encrypts each entry, producing ciphertext wrapped in ciphertext, written to both the server and the device. On the next unlock, one decrypt returns the old ciphertext, so every entry renders as unreadable text, permanently.

Fix Make the reseal refuse to run unless the entries were actually decrypted under the old key — that is, abort when the store was hydrated while locked. The deeper fix is to make the sealed state a real field on the entry rather than re-deriving it from whether the vault happens to be unlocked at write time.

High

H1

The two-factor fix undoes itself the first time anyone adds a wallet HIGH

NEW verified

20260808010000_wallet_schema_self_heal.sql:105-107 · 20260904010000_mfa_rls_hardenning.sql:27-30

The policy fix is correct, and any user can erase it with one ordinary action. Creating a custom wallet triggers a rebuild that loops over every registered wallet and re-runs the table factory on each. The password vault is a registered wallet, so the factory drops the four policies — by the same names the fix uses — and recreates them without the two-factor condition.

The result is that one user tapping "Add a wallet" silently removes two-factor enforcement from the password vault for the entire project, permanently, until someone re-runs the migration by hand.

Fix Have the table factory skip the password vault, or re-apply the condition at the end of every rebuild. The fix and the thing that dissolves it currently share four identifiers.

H2

Anyone can delete family-vault documents with no account at all HIGH

verified

20260734000000_fix_vault_document_audit.sql:86, :106

Long-standing, surfaced this pass. The remove-document function is granted to authenticated but never revoked from PUBLIC, and granting to one role does not remove the default grant to everyone — so it is callable with the anon key that ships in the app.

Its authorisation check then fails open. For a caller with no session the ownership comparison yields null rather than false, null combined with false is null, and the negation of null is still null — which the database treats as "do not enter the branch". The error is never raised and the delete proceeds. Someone removed from a shared vault who kept the document ids can wipe that vault's document index from outside any session.

Fix Revoke from PUBLIC and add an explicit null check on the caller before the comparison. This three-valued-logic trap is worth grepping for across the other definer functions.

H3

None of the backend work can currently reach the database DEPLOYMENT

three duplicate migration timestamps · 20260904040000:224-231

Two blockers compound. Three pairs of migrations share identical timestamp prefixes, and the migration table keys on that value, so a CLI push collides on the second of each pair. Separately, the storage quota trigger still selects a column that does not exist on the storage table, outside any error handler, which would reject every document upload; and its trigger creation is unguarded, so a permission error rolls back the whole migration including the wallet and invite fixes in the same file.

The practical consequence is that everything verified in the backend section above is currently paper-only unless each file is pasted into the SQL editor by hand.

Fix Rename one file in each pair by one second, and correct the quota query to read the size out of the metadata field as the project's own working helper already does.

H4

iOS deep links cannot work, and the test says they can HIGH verified

apple-app-site-association:6 · share_security_hardening_test.dart:143

See the panel above. The association file carries the literal string `TEAMID`, no team ID exists in the Xcode project, and the guarding test uses a substring match that the placeholder satisfies.

H5

The whole share website shares one rate-limit budget HIGH

share-frontend/app/api/**/route.ts · functions/share/index.ts:144-153

Worse than reported last time. None of the five web proxy routes forward the visitor's IP, so every web visitor resolves to one address. The password lockout keys on address plus share, so five wrong guesses by anyone locks out every web visitor to that share. But the request limiter keys on address alone — so the entire share site shares a single sixty-per-minute budget across all shares and all visitors, and ordinary traffic can lock the site out of itself.

The mobile app talks to the backend directly, so its addresses are real. That is why this keeps surviving app-side testing.

Also open

AREA

ISSUE

SEVERITY

Backend	The invite function still writes the target's name, email and phone into a row the caller reads straight back, and a constraint violation on name-only lookups gives a clean second oracle.	High
Backend	Account deletion still destroys co-owners' vaults with no transfer, and removes registry rows for custom wallets other users store data in.	Medium
Backend	Pre-existing bad vault-document rows are still not cleaned up, so old cross-user file access persists despite the write-path trigger.	Medium
Backend	The lockout never decays, so one user who fumbles five times is throttled indefinitely; the table has no purge and stores recipient IPs in plaintext while every other table hashes them.	Medium
Backend	Device-token handover is still broken from both ends, so a resold phone keeps receiving the previous owner's notifications.	Medium
Crypto	Every account's offline library is still wiped on every account switch, contradicting the file's own documented contract.	Medium
Crypto	Decrypted documents are still written to the temp root and never deleted — the cleanup targets a different directory, partly defeating the new at-rest encryption.	Medium
Crypto	The vault key is still destroyed by any OS dialog, from two separate observers, one of which fires even when the app lock is off. This is also what makes the Critical above reachable.	Medium
Auth	The two-factor screen still stacks twice on the account-switch path and still has no back-button guard, and the check still fails open through a flag cleared moments earlier.	Medium
Auth	"Sign out other devices" still revokes no token; a stolen device kept offline stays signed in.	Medium
Auth	The new session storage falls back to writing plaintext if Keystore throws, silently reverting to the old posture on affected devices.	Medium
Privacy	No legal URL is asserted live, and the documents, app config, entitlements and README now name four different hosts. The README correctly calls the documented one dead.	Play blocker
Privacy	The in-app policy is still six thin sections dated July 2026, with no processors, retention, statutory rights, grievance contact or link to the published version.	High

Privacy	The web deletion page still has no form and no API route behind it, while the document describes a verification-code flow that does not exist.	High
Privacy	The export still returns a masked ID beside the raw one in the notes field, and covers three of roughly fifteen data domains.	Medium
Privacy	Cloud speech is still the default with no caller requesting on-device; creating a reminder still silently re-registers push after the user turned it off.	Medium
Platform	CI still publishes a debug-signed APK as the latest public release through six unpinned actions with write access; the correct procedure sits documented but unimplemented.	Medium
Platform	Still no data-extraction rules under targetSdk 36, so device-to-device transfer is ungoverned; iOS still compiles out the microphone the app requests.	Medium

A note on the privacy surface

One finding is worth stating plainly because it explains why that score moved only one point. Running the history over the thirteen flagged privacy paths — the policy documents, the localisation table, the legal screens, the export service, the voice service, the reminder scheduler, the deletion page and the web API directory — across all five commits returns nothing at all. That surface was not touched.

The consent gate was fixed, and fixed well. Everything else in that area is unchanged by construction, including both remaining Play blockers. If the goal is submission rather than hardening, that is where the next round belongs.

What to do next

1 Stop the reseal from running on a locked store

One condition. Until then, "Forgot passphrase" silently destroys every saved password, and the honest count makes users more likely to confirm it.

- 2 **Revoke the remove-document function from PUBLIC and add a null-caller check**
Two lines. Closes unauthenticated deletion of vault documents. Then grep the other definer functions for the same three-valued-logic trap.

- 3 **Stop the wallet rebuild from stripping the two-factor policies**
Skip the password vault in the table factory, or re-apply the condition after each rebuild.

- 4 **Unblock deployment**
Rename one file in each of the three duplicate timestamp pairs, and fix the quota query's column. Without this none of the backend work can reach the database through the CLI.

- 5 **Put your real Apple Team ID in the association file**
And change the test to assert the full identifier rather than a substring, so it can fail.

- 6 **Forward the client IP through the web proxies**
Otherwise the share site rate-limits itself as a whole.

- 7 **Then spend a round entirely on privacy**
Pick one domain and deploy it, give the deletion page a real request path, and rewrite the in-app policy to match the published one. These are the two remaining Play blockers.

Items 1 through 6 are small and mostly single-line. With those and item 7 done, this is a 8 out of 10 and a submission that should survive review. The engineering is converging; the compliance surface is the part that has not started moving.

Suggested change to how fixes are checked

Writing tests this round was the right instinct. The refinement worth making is to assert the property you care about rather than the presence of the change. Three concrete examples from this report: assert the association file equals your full team identifier rather than containing the bundle id; assert that a caller with no session is *refused* by the remove-document function rather than that the function exists; and assert that a locked store refuses to reseal rather than that a guard was added.

Each of the three new findings above would have been caught by a test written that way, and all three passed the suite as written.

Method: four parallel re-audits covering backend and row-level security, auth and session, cryptography and local data, sharing and platform, and privacy and Play

compliance. Every Critical and High was independently re-read in the code before inclusion, with several agent claims checked against the defining schema and confirmed on that basis. The Android build was verified by running the manifest merger, the regressing paths were confirmed by walking the commit history, and the full test suite was run at 508 of 508 passing. No files were modified during this audit.